



What is a Service in Kubernetes?

A **Service** is an abstraction that defines a **logical set of Pods** and a **policy to access them**. Since Pods are ephemeral (they can be created and destroyed), Services provide **stable network access** to them.

♦ Types of Services in Kubernetes

1. ClusterIP (default)

- Accessible only inside the cluster
- Assigns an internal IP address.
- Good for **internal communication** between services (e.g., frontend talks to backend).

Use case: Backend services, internal APIs

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

2. NodePort

- Exposes the Service on **each Node's IP** at a static port (e.g., 30000–32767).
- You can access it via **<NodeIP>:<NodePort>**.

Use case: Quick way to expose an app without a cloud Load Balancer.

```
apiVersion: v1
kind: Service
metadata:
  name: my-nodeport-service
spec:
  type: NodePort
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
      nodePort: 30007
```

3. LoadBalancer

- Provisioned by cloud providers (e.g., AWS, GCP, Azure).
- Exposes the Service externally using a **cloud load balancer**.
- Internally uses a NodePort or ClusterIP.

Use case: Public-facing services in cloud environments.

```
apiVersion: v1
kind: Service
metadata:
  name: my-loadbalancer-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - port: 80
      targetPort: 8080
```

 Cloud provider assigns an external IP address.

4. ExternalName

- Maps a Service to an **external DNS name**.
- No selector or Pod access—just DNS redirection.

Use case: Access external services (e.g., a third-party database or API) via an internal Service name.

```
apiVersion: v1
kind: Service
metadata:
  name: my-external-service
spec:
  type: ExternalName
  externalName: example.com
```

 When Pods access `my-external-service`, it resolves to `example.com`.

5. Headless Service (**ClusterIP: None**)

- Does **not assign a ClusterIP**.
- Used for **direct Pod-to-Pod communication**, service discovery, and **stateful applications**.

Use case: StatefulSets, custom load balancing, DNS lookups of individual Pods.

```
apiVersion: v1
kind: Service
metadata:
  name: my-headless-service
spec:
  clusterIP: None
  selector:
    app: my-app
  ports:
    - port: 80
```

 DNS returns the IPs of individual Pods instead of a single ClusterIP.

Summary Table

Service Type	Exposed To	Use Case
ClusterIP	Internal only	Internal services
NodePort	External via Node IP	Dev/test, no cloud LB needed
LoadBalancer	External (Cloud)	Public access in cloud setups
ExternalName	External DNS redirect	Third-party services
Headless	Internal (no ClusterIP)	Stateful apps, custom discovery
